PARTE 3 DE 4 · 3-4 MINUTOS

Jose Vasquez

Lógica de negocio: servicios, reglas y excepciones

Tu parte es **el cerebro del sistema**: los servicios. Explicas su estructura y, sobre todo, **las reglas de negocio**. La matrícula es lo más importante: dale más tiempo. Termina con las excepciones, porque Isaac parte de ahí.

Orden de la exposición

#	Quién	Tema	Tiempo
1	Santiago Arango	Introducción, tecnologías y arquitectura	3 min
2	Alex Monroy	Modelo de datos: entidades, relaciones y repositorios	3 min
3	Jose Vasquez	Lógica de negocio: servicios, reglas y excepciones	3-4 min
4	Isaac	API REST, validaciones, errores y demo en vivo	4-5 min

Antes de empezar

- Abiertos: **EnrollmentServiceImpl.java**, **StudentServiceImpl.java** y la carpeta **domain/Exceptions**.

Guion paso a paso

Cada paso dice qué mostrar en pantalla y qué decir. No lo leas palabra por palabra: úsalo como guía y dilo con tus palabras.

1. Estructura de los servicios

Muestra: EnrollmentServiceImpl.java, la parte de arriba de la clase.

```
@Service
@Transactional
public class EnrollmentServiceImpl implements EnrollmentService { ... }
```

QUÉ DECIR

Cada servicio tiene una interfaz con las operaciones y una clase Impl que las implementa; el controlador solo conoce la interfaz. Con @Service, Spring crea el objeto y lo inyecta por el constructor. Y @Transactional hace que cada operación sea todo o nada: si algo falla, no queda nada guardado a medias.

2. Reglas de estudiantes y cursos

Muestra: StudentServiceImpl.create

```
if (studentRepository.existsByEmail(student.getEmail())) {
    throw new StudentEmailAlreadyExistsException(student.getEmail());
}
```

QUÉ DECIR

Antes de guardar un estudiante revisamos que el correo no exista, al crear y también al editar. Con los cursos pasa lo mismo con el código, y además no se puede bajar el cupo por debajo de los estudiantes activos que ya tiene.

3. Crear una matrícula (lo más importante)

Muestra: EnrollmentServiceImpl.create. Señala cada bloque.

```
Student student = studentRepository.findById(studentId)
        .orElseThrow(() -> new StudentNotFoundException(studentId));    // 1
Course course = courseRepository.findById(courseId)
        .orElseThrow(() -> new CourseNotFoundException(courseId));      // 2
if (enrollmentRepository.existsByStudent_IdAndCourse_IdAndStatusNot(
        studentId, courseId, EnrollmentStatus.CANCELLED)) {
    throw new ResourceConflictException("...");                         // 3
}
checkAvailableSpots(course);                                            // 4
return enrollmentRepository.save(new Enrollment(..., ACTIVE));         // 5
```

QUÉ DECIR

Para matricular hacemos cinco pasos: que el estudiante exista, que el curso exista, que no esté ya inscrito en ese curso (las canceladas no cuentan, así puede volver a inscribirse), que haya cupo, y si todo pasa se guarda como activa, con la fecha de hoy si no nos enviaron otra.

4. Cambios de estado

Muestra: EnrollmentServiceImpl.updateStatus

De	A	¿Se permite?
ACTIVE	CANCELLED o COMPLETE	Sí
CANCELLED	ACTIVE	Sí, si hay cupo y no tiene otra matrícula vigente
CANCELLED	COMPLETE	No
COMPLETE	Cualquiera	No, ya terminó el curso

QUÉ DECIR

Una matrícula activa se puede cancelar o completar. Una cancelada se puede reactivar, pero vuelve a pasar por las validaciones. Y una completada ya no se modifica.

5. Excepciones

Muestra: La carpeta domain/Exceptions.

```
BusinessException                    -> 400
 ├── ResourceNotFoundException       -> 404  (Student/Course/EnrollmentNotFound)
 └── ResourceConflictException       -> 409  (StudentEmailAlreadyExists)
```

QUÉ DECIR

Cuando una regla no se cumple, el servicio lanza una excepción propia. Están organizadas por herencia, y cada tipo se convierte en un código HTTP distinto, que es lo que Isaac les va a mostrar.

Preguntas que te pueden hacer

Si la base de datos ya tiene unique en el correo, ¿por qué validarlo en el servicio?

Para responder un 409 con un mensaje claro en vez de un error de base de datos. El unique queda como respaldo.

¿Qué pasa si dos personas toman el último cupo al mismo tiempo?

La transacción ayuda, pero con mucha concurrencia podría pasar. Se resolvería con un bloqueo pesimista; lo dejamos como mejora.

¿Por qué inyección por constructor?

Es lo que recomienda Spring: los campos quedan final y las dependencias son explícitas.

Cierre y paso al siguiente

QUÉ DECIR

Esas son las reglas del sistema. Ahora Isaac les muestra cómo se expone todo como API y lo vemos funcionando en vivo.

Consejos

- Tu parte tiene más código: explica la idea de cada bloque, no lo leas línea por línea.
- Ensayen una vez los cuatro seguidos con cronómetro. Si alguien se pasa, el que sufre es la demo del final.
- Cuando muestren código, hagan zoom en IntelliJ (Ctrl + rueda del mouse) para que se lea desde atrás.
- Si no saben una respuesta, digan qué parte del código la resuelve o que lo revisan; es mejor que inventar.